Call Gates

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

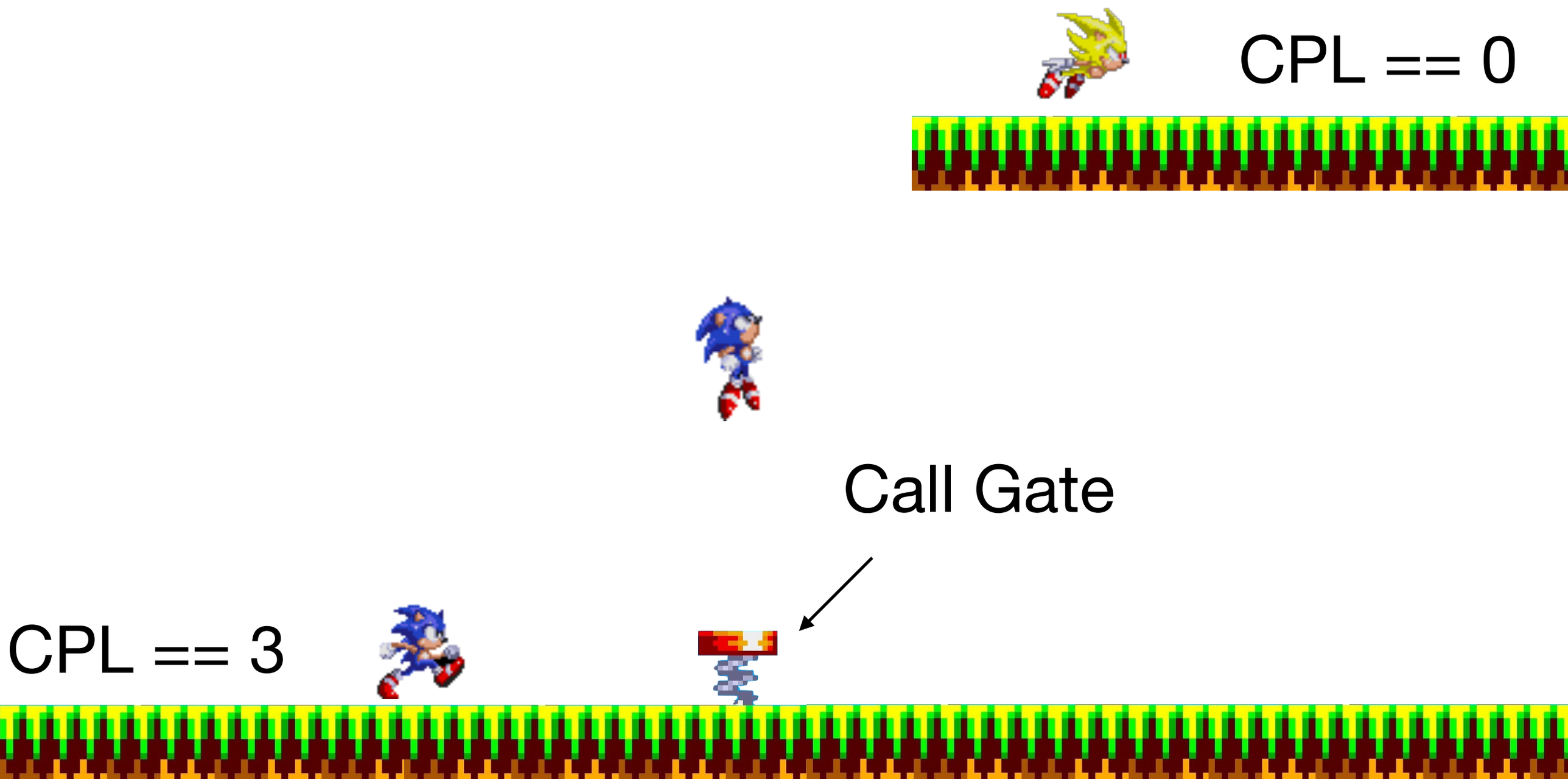
Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 2001: x86-64 OS Internals’ class, available at https://ost.fyi"

Call Gates

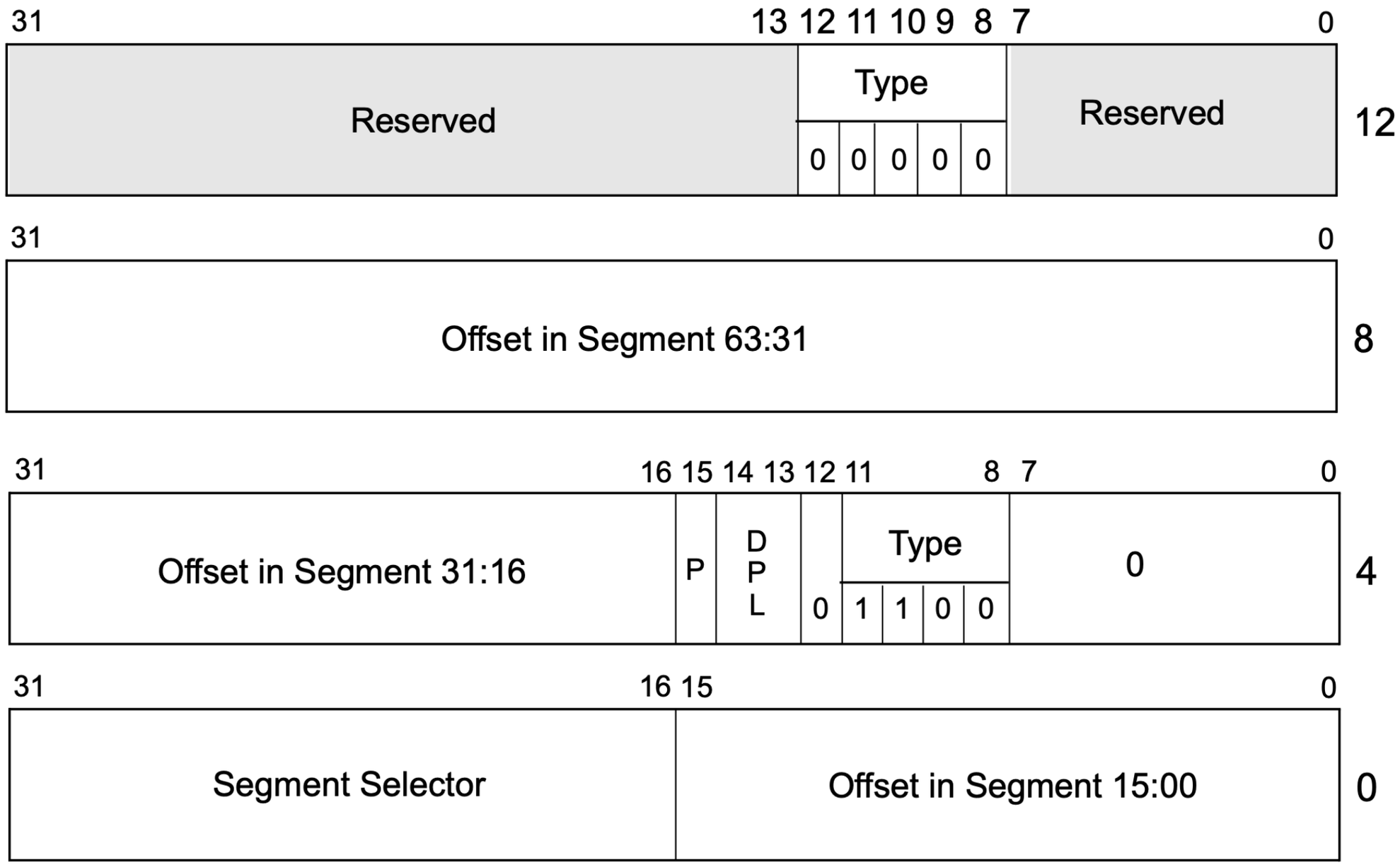
- A *Call Gate* is a way to transfer control from one segment to another segment at a different privilege level

Call Gates

- A *Call Gate* is a way to transfer control from one segment to another segment at a different privilege level



Call Gate Descriptor



DPL Descriptor Privilege Level
P Gate Valid

Figure 5-9. Call-Gate Descriptor in IA-32e Mode

Table 3-2. System-Segment and Gate-Descriptor Types

Type Field					Description	
Decimal	11	10	9	8	32-Bit Mode	IA-32e Mode
0	0	0	0	0	Reserved	Reserved
1	0	0	0	1	16-bit TSS (Available)	Reserved
2	0	0	1	0	LDT	LDT ✅
3	0	0	1	1	16-bit TSS (Busy)	Reserved
4	0	1	0	0	16-bit Call Gate	Reserved
5	0	1	0	1	Task Gate	Reserved
6	0	1	1	0	16-bit Interrupt Gate	Reserved
7	0	1	1	1	16-bit Trap Gate	Reserved
8	1	0	0	0	Reserved	Reserved
9	1	0	0	1	32-bit TSS (Available)	64-bit TSS (Available)
10	1	0	1	0	Reserved	Reserved
11	1	0	1	1	32-bit TSS (Busy)	64-bit TSS (Busy)
12	1	1	0	0	32-bit Call Gate	64-bit Call Gate ✅
13	1	1	0	1	Reserved	Reserved
14	1	1	1	0	32-bit Interrupt Gate	64-bit Interrupt Gate
15	1	1	1	1	32-bit Trap Gate	64-bit Trap Gate

Call the Call Gate

- To transition from CPL 3 to CPL 0, you would issue a *CALL* instruction, with a *far pointer*, that had a *Segment Selector*, that pointed at a Call Gate *Segment Descriptor*
- It's so clear right? 😅

Faux "Far Pointer" to Far Pointer!

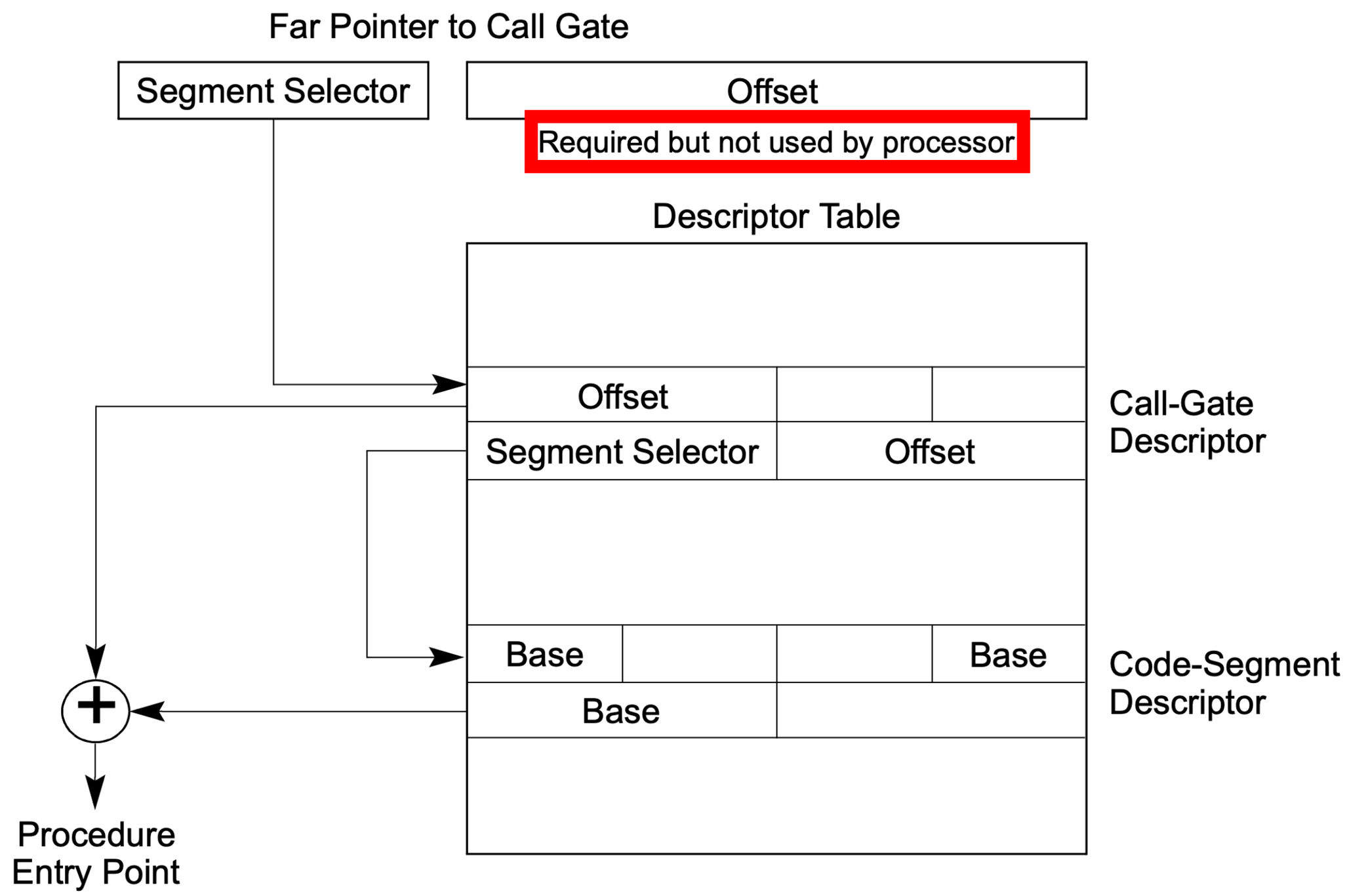


Figure 5-10. Call-Gate Mechanism

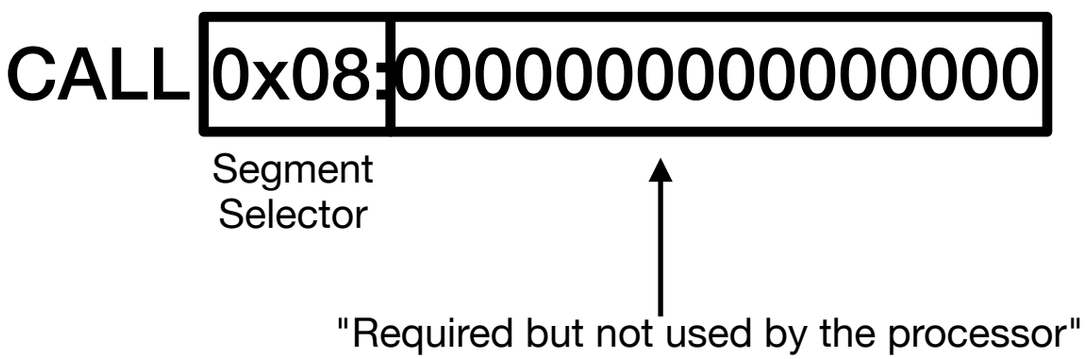
Calling All Call Gates!

Logical Address
(Faux) Far Pointer

CALL 0x08:0000000000000000

Calling All Call Gates!

Logical Address
(Faux) Far Pointer



Calling All Call Gates!

CALL 0x08:0000000000000000

Calling All Call Gates!

CALL 0x08:0000000000000000

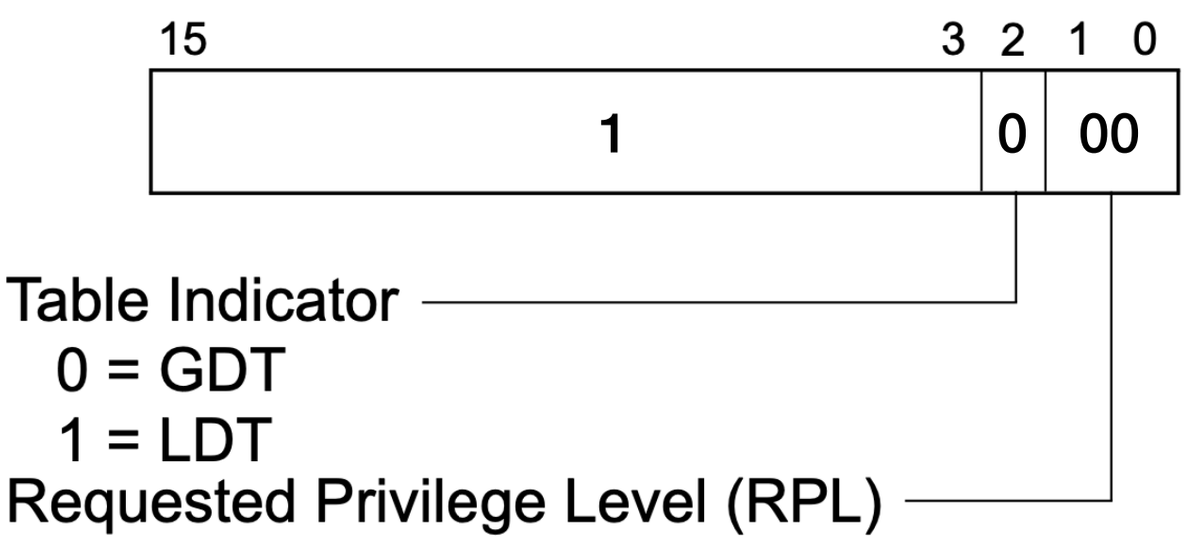


Figure 3-6. Segment Selector

Calling All Call Gates!

CALL 0x08:0000000000000000

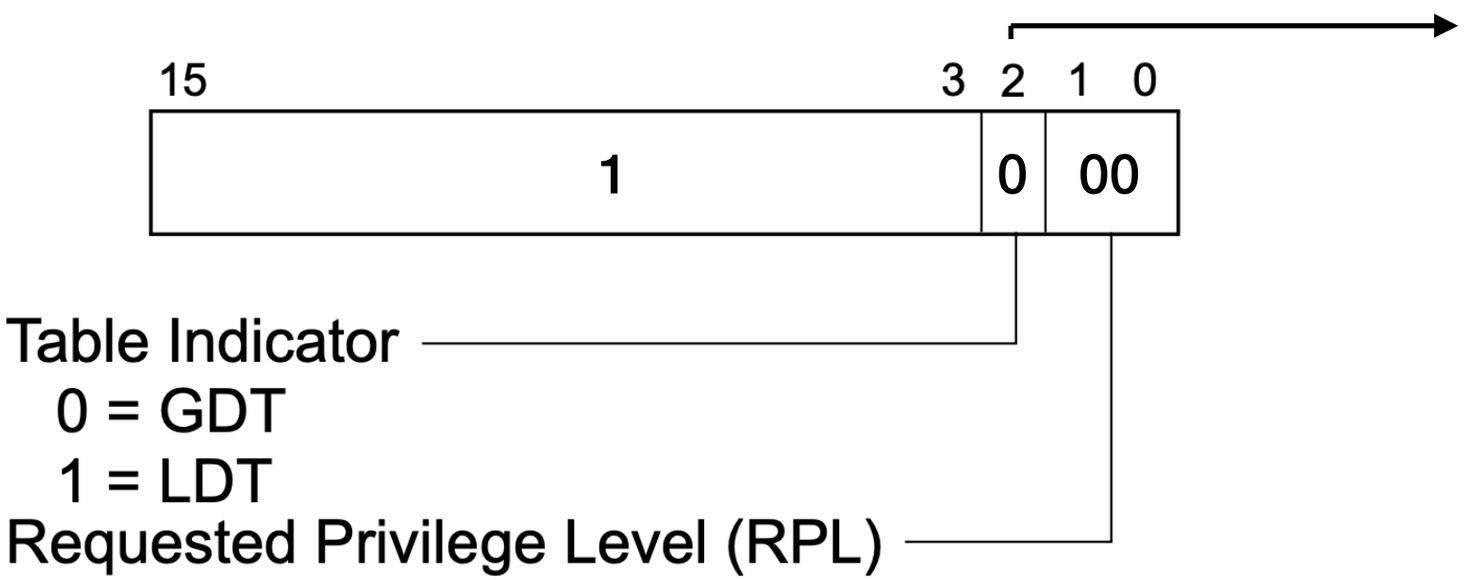


Figure 3-6. Segment Selector

Calling All Call Gates!

CALL 0x08:0000000000000000

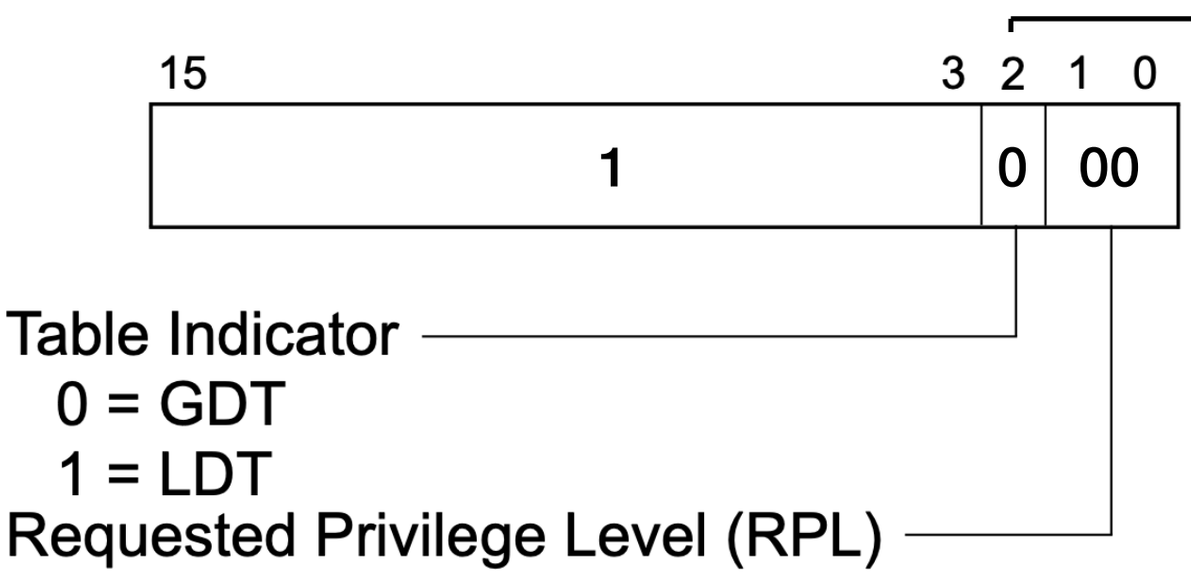


Figure 3-6. Segment Selector

Calling All Call Gates!

CALL 0x08:0000000000000000

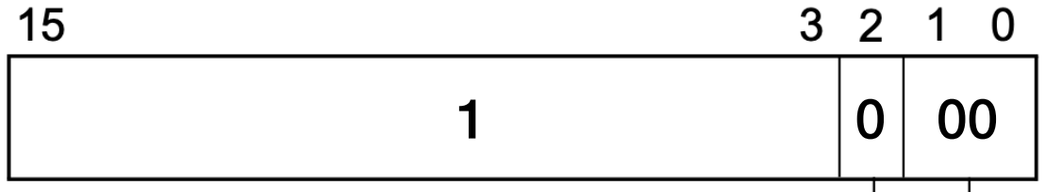


Table Indicator
0 = GDT
1 = LDT
Requested Privilege Level (RPL)

Figure 3-6. Segment Selector

Calling All Call Gates!

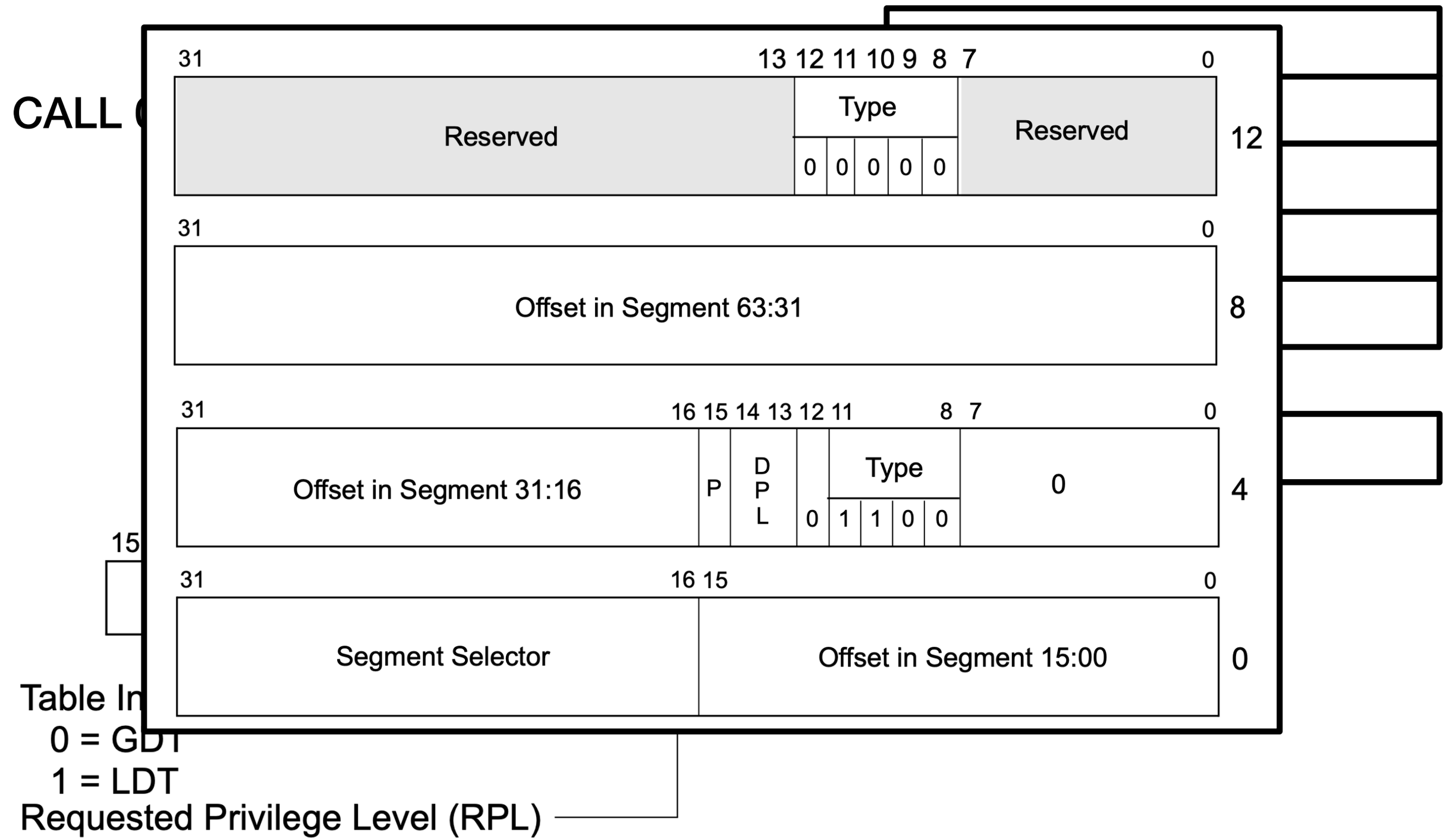


Figure 3-6. Segment Selector

Calling All Call Gates!

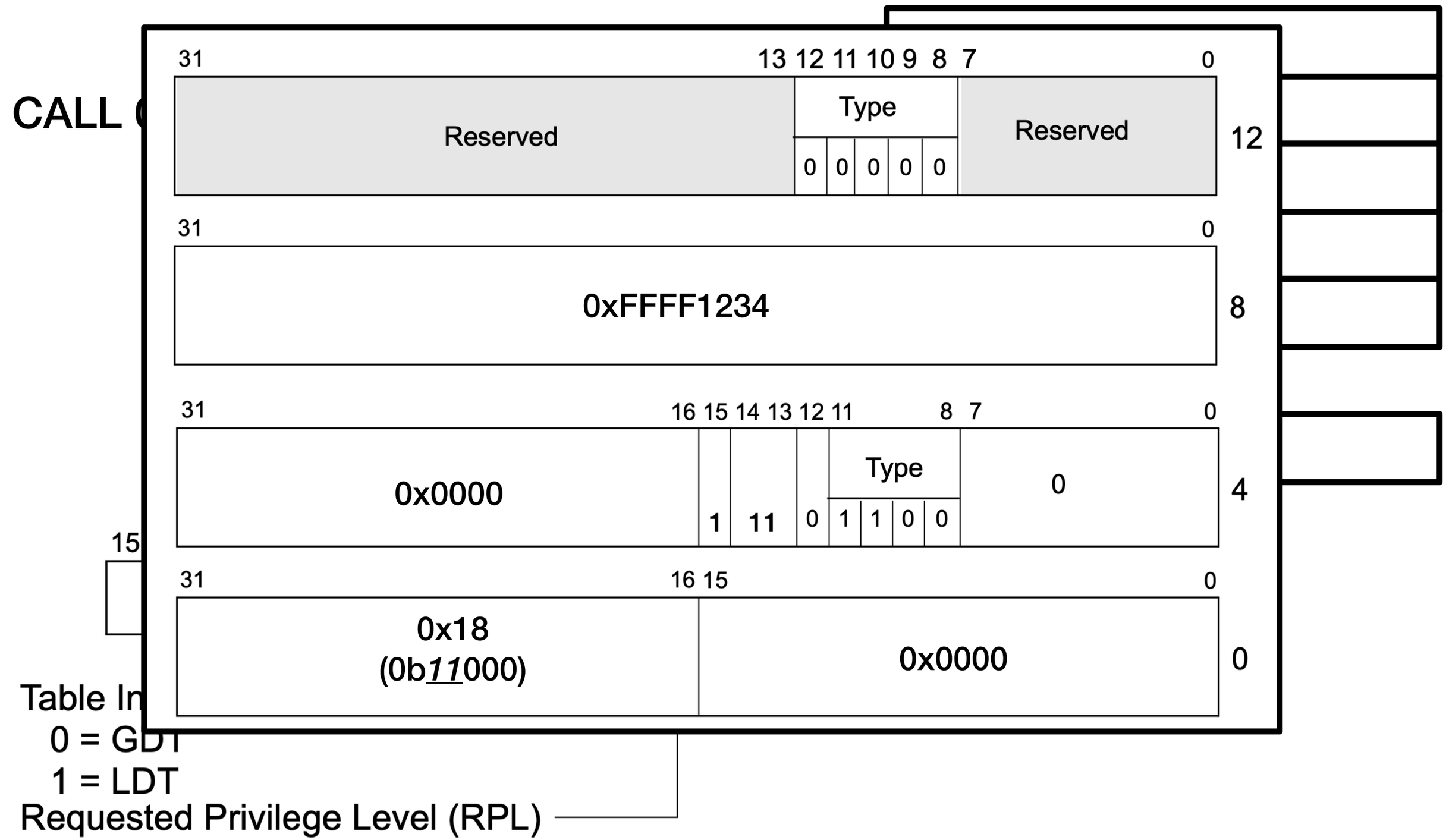


Figure 3-6. Segment Selector

Calling All Call Gates!

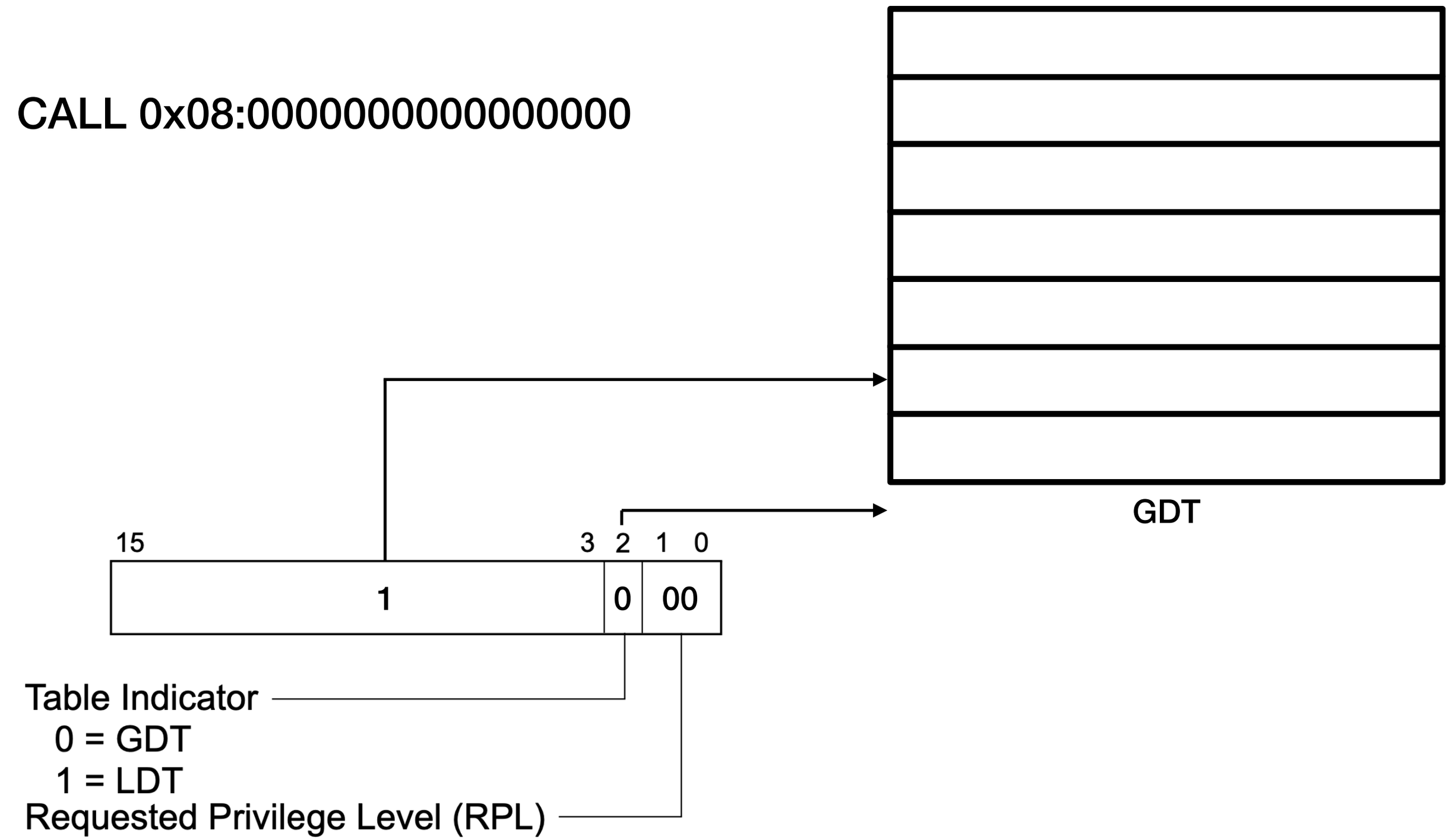


Figure 3-6. Segment Selector

Calling All Call Gates!

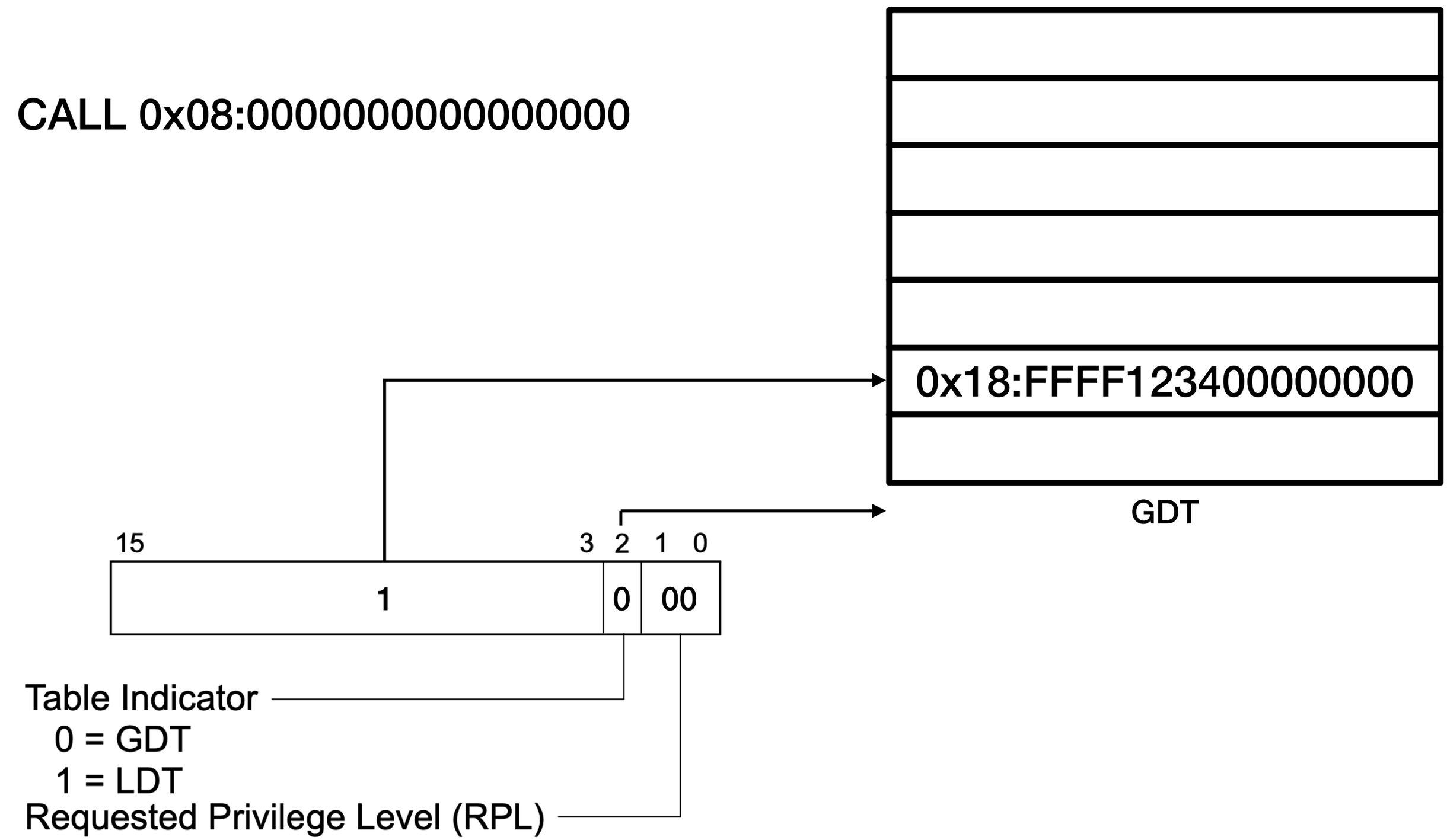


Figure 3-6. Segment Selector

Calling All Call Gates!

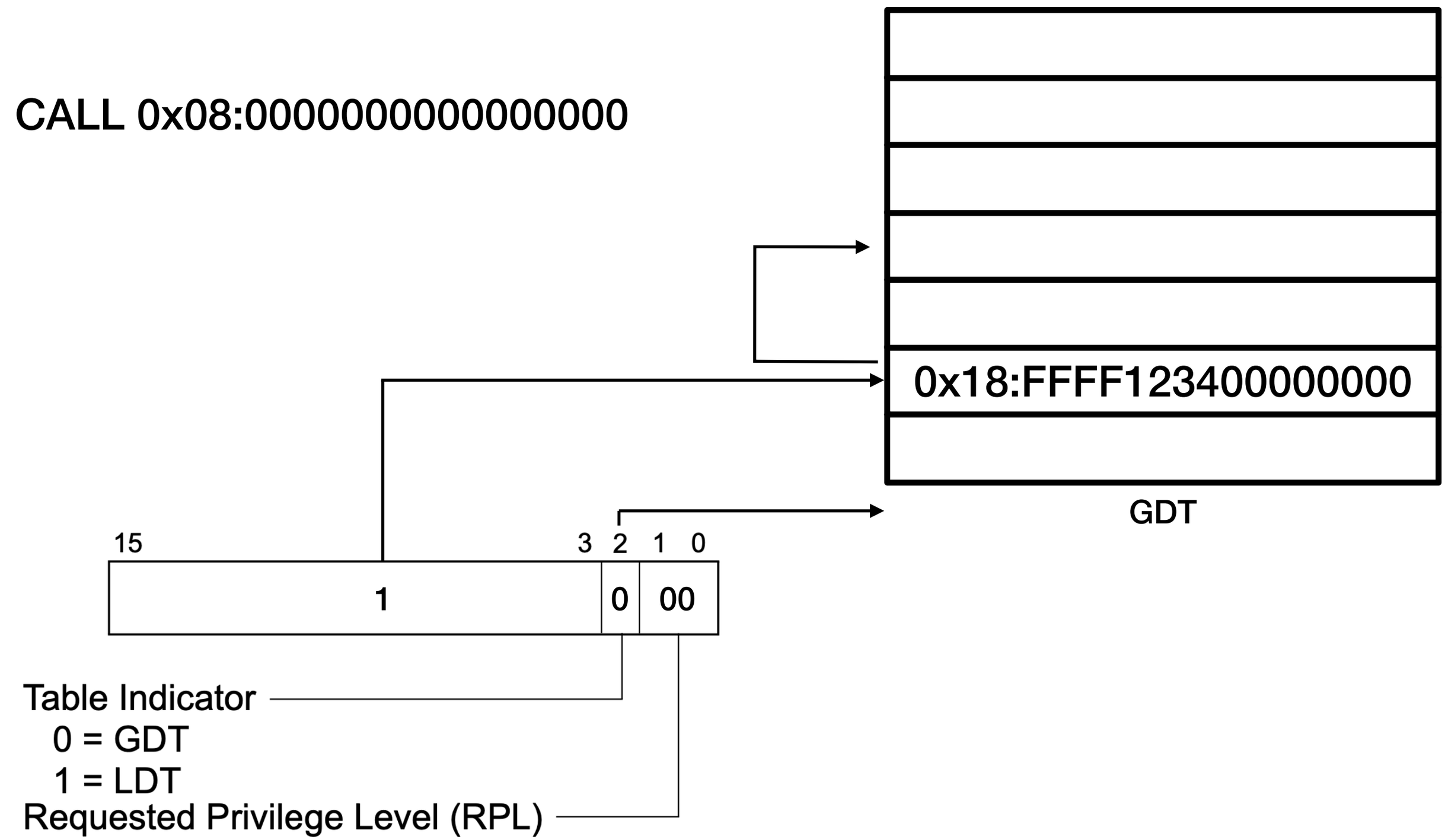


Figure 3-6. Segment Selector

Calling All Call Gates!

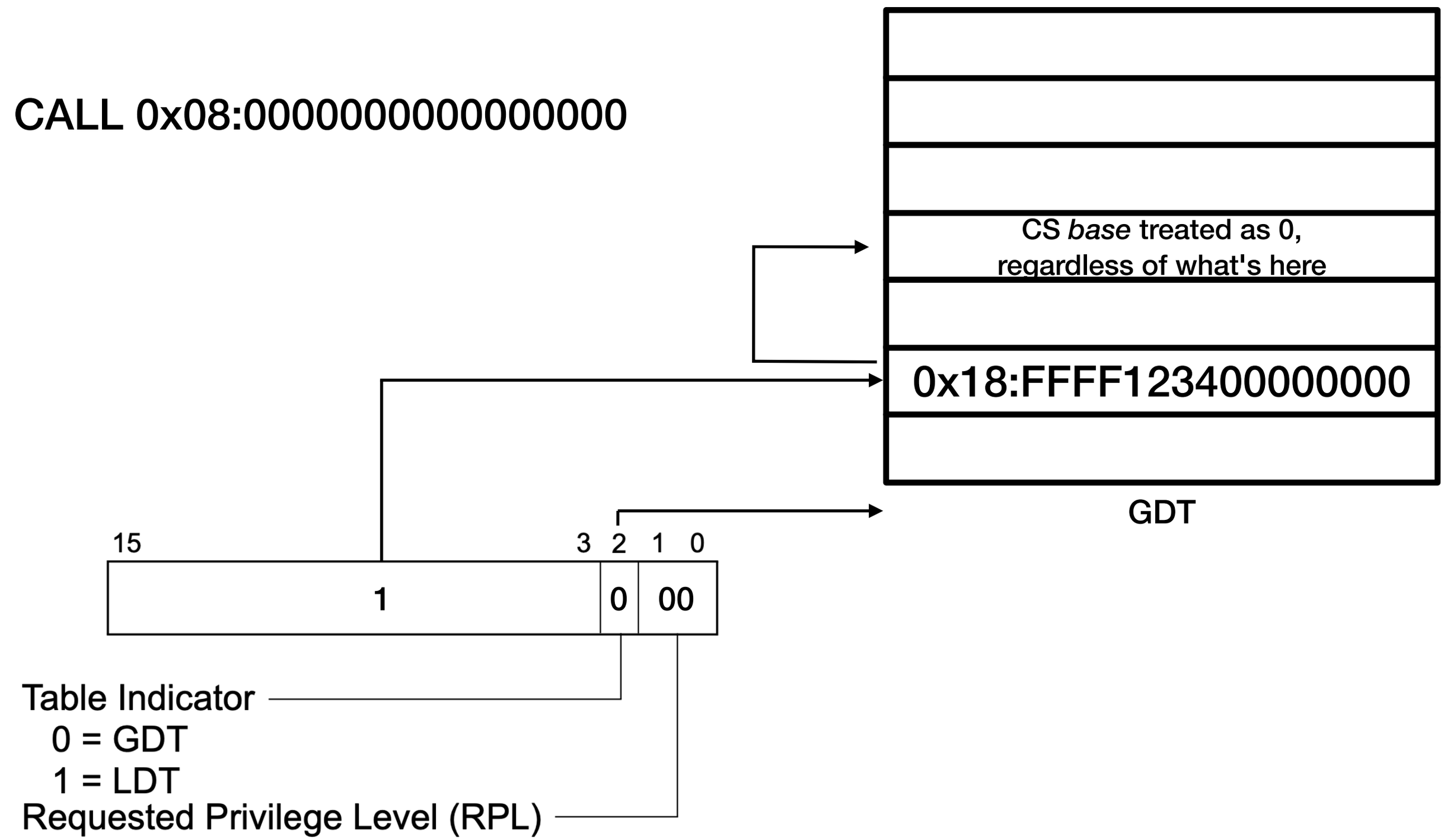


Figure 3-6. Segment Selector

Calling All Call Gates!

CALL 0x08:0000000000000000

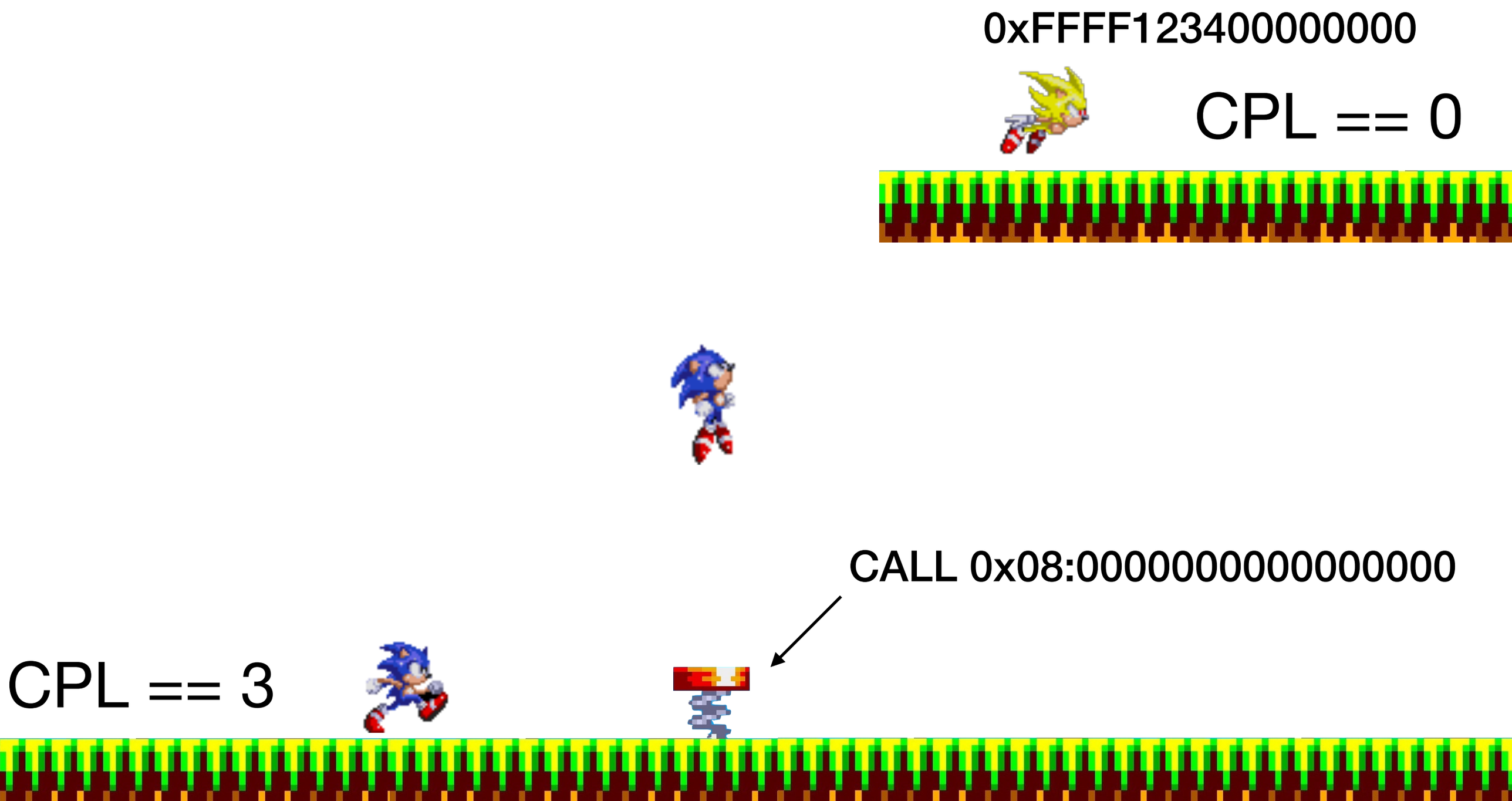
0x18:	0 (base) + FFFF123400000000
New CS	**New RIP**

Calling All Call Gates!

CALL 0x08:0000000000000000

0xFFFF123400000000

Boost!


0xFFFF123400000000
CPL == 0
CALL 0x08:0000000000000000
CPL == 3

How Can I Return From a Call Through a Call Gate?

- A normal CALL pushes RIP
- A inter-privilege far CALL through a Call Gate pushes SS:RSP and CS:RIP
- RET can pop those back off the stack to return from an inter-privilege far call

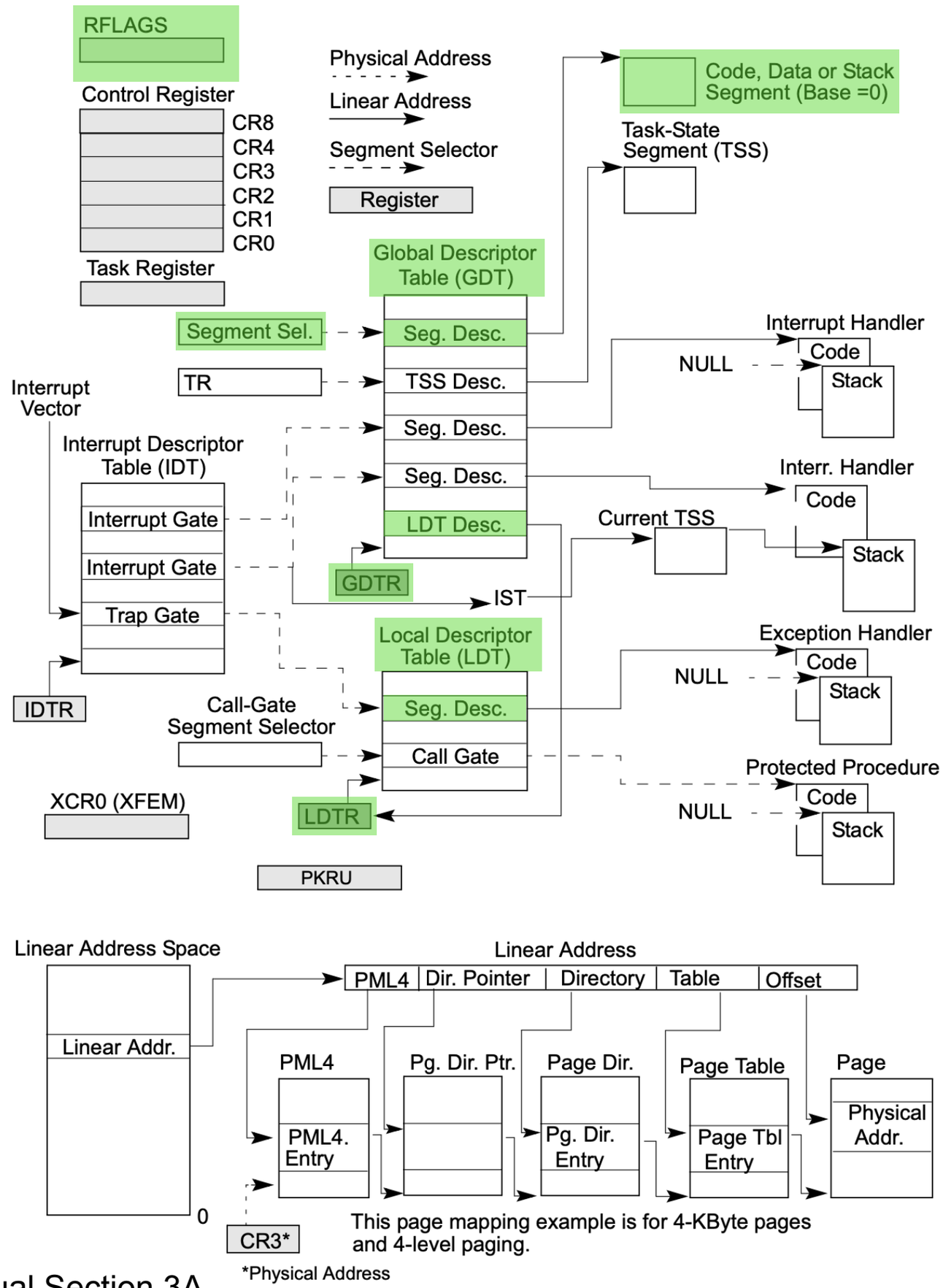
Table 5-2. 64-Bit-Mode Stack Layout After Far CALL with CPL Change

32-bit Mode			IA-32e mode		
Old SS Selector	+12			+24	Old SS Selector
Old ESP	+8			+16	Old RSP
CS Selector	+4			+8	Old CS Selector
EIP	0	**ESP**	**RSP**	0	RIP
< 4 Bytes >					< 8 Bytes >

Vol 3, Sect 5.8.5.1

Call Gates 4

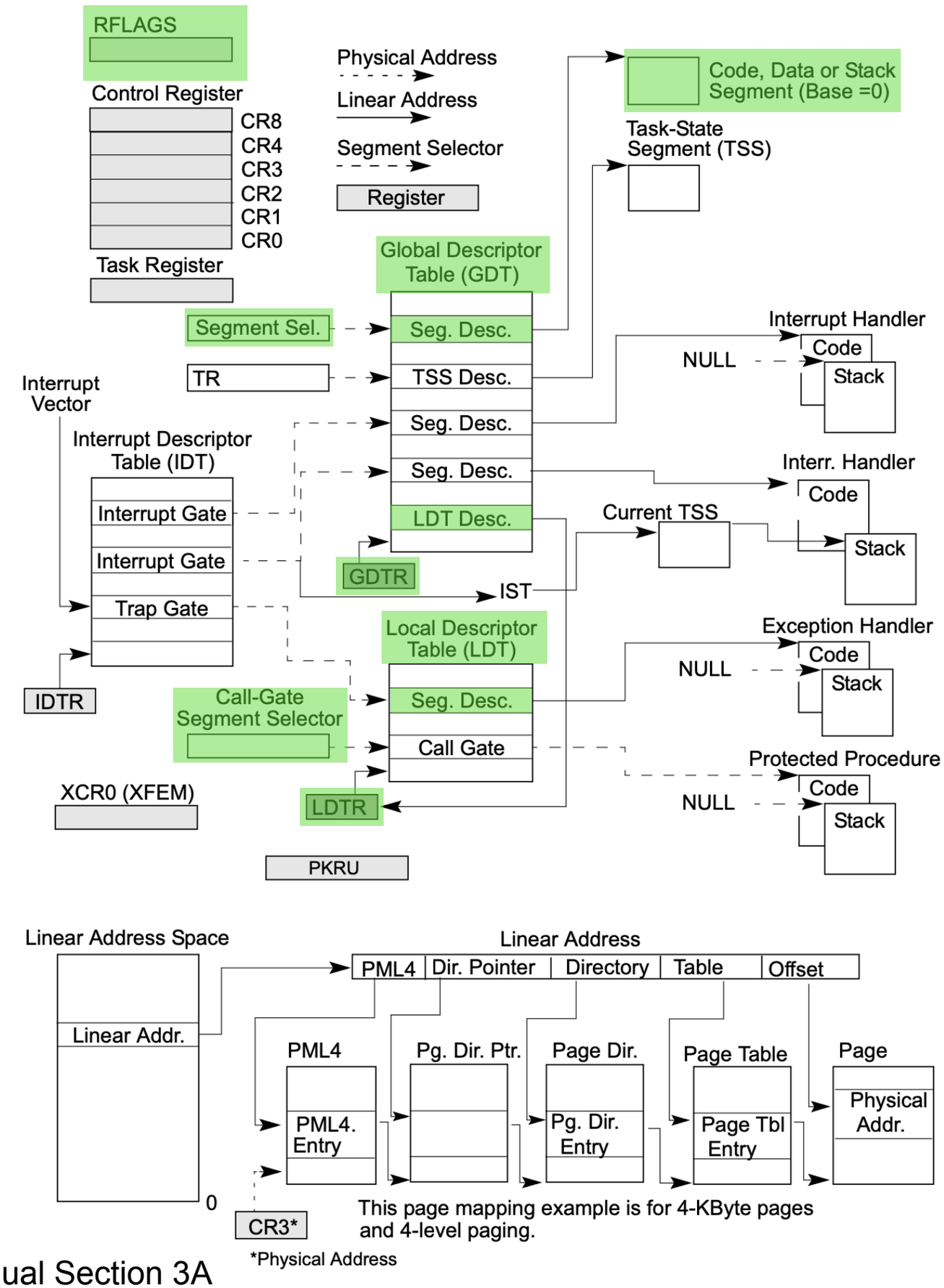
- *Call gates don't tend to be used these days*
- I just wanted you to see the first notion of privilege escalation that Intel baked into their architecture, before we work our way towards what *is* used
  - Also again, if you check out the blog post I referenced about the LDT on Windows, you'll see how Alex Ionescu found a way to use call gates for an attack (albeit after it was already disabled)
  - https://www.alex-ionescu.com/?p=340



Intel Manual Section 3A

That's what you're going to learn! :D

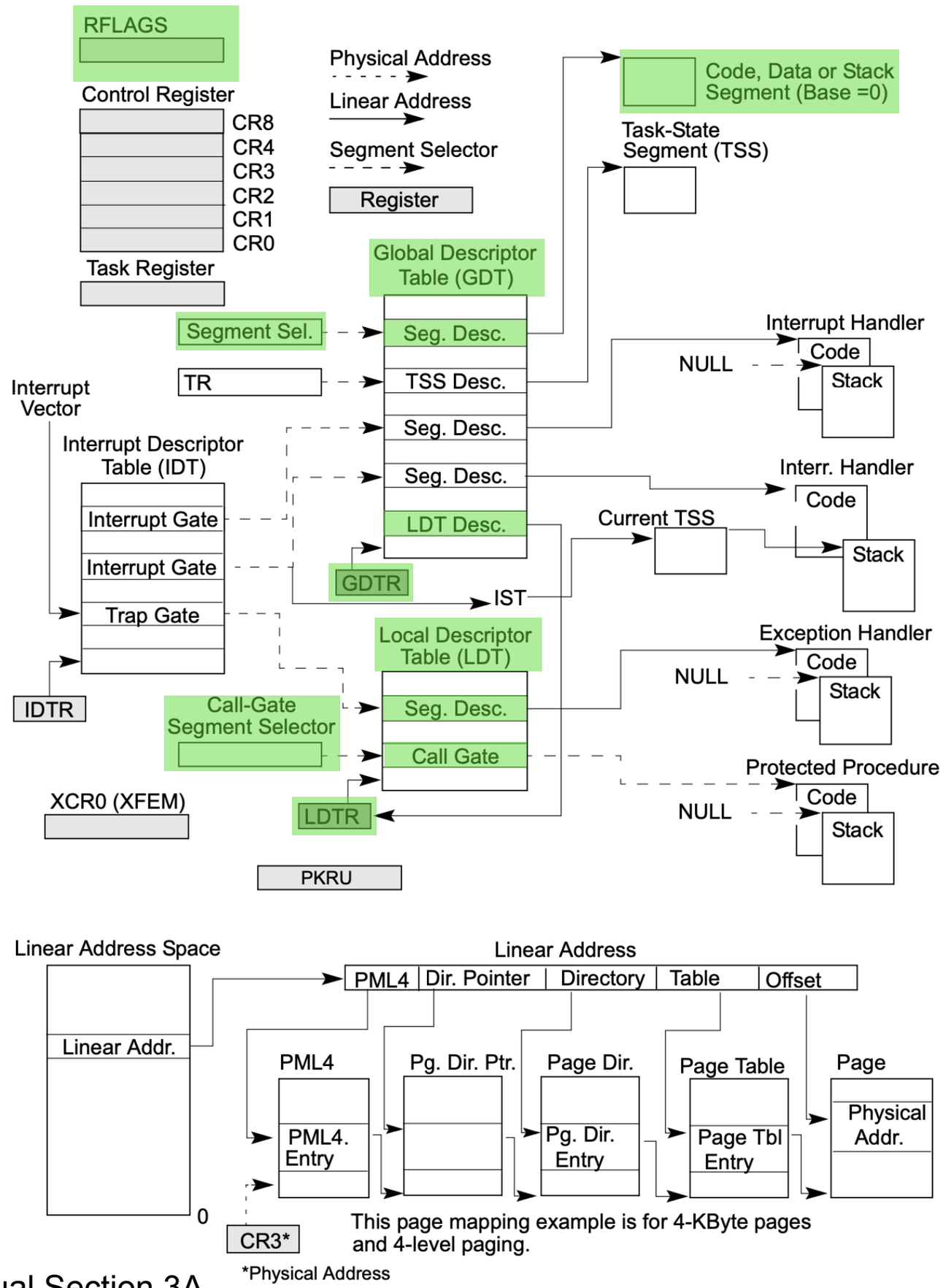
Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Intel Manual Section 3A

That's what you're going to learn! :D

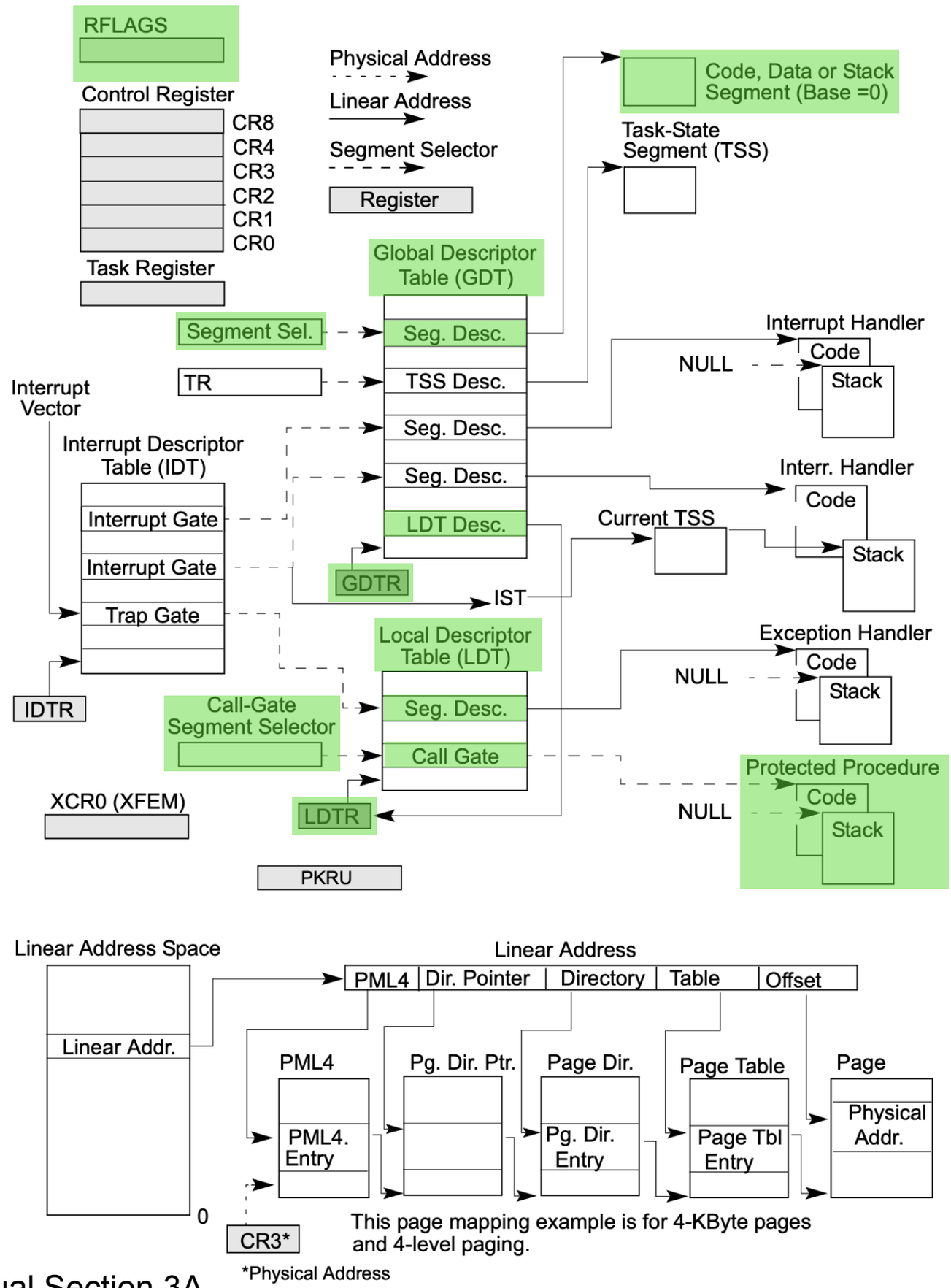
Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Intel Manual Section 3A

That's what you're going to learn! :D

Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Intel Manual Section 3A

That's what you're going to learn! :D

Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging